Counter cache columns are added to the containing model's list of read-only attributes through `attr_readonly`.

4.1.2.4 `:dependent`

If you set the `:dependent` option to:

- `:destroy`, when the object is destroyed, `destroy` will be called on its associated objects.
- `:delete`, when the object is destroyed, all its associated objects will be deleted directly from the database without calling their `destroy` method.

You should not specify this option on a `belongs_to` association that is connected with a `has_many` association on the other class. Doing so can lead to orphaned records in your database.

4.1.2.5 `:foreign_key`

By convention, Rails assumes that the column used to hold the foreign key on this model is the name of the association with the suffix `_id` added. The `:foreign_key` option lets you set the name of the foreign key directly:

```
class Order < ActiveRecord::Base
  belongs_to :customer, class_name: "Patron",
                        foreign_key: "patron_id"
end
```

In any case, Rails will not create foreign key columns for you. You need to explicitly define them as part of your migrations.

4.1.2.6 `:inverse_of`

The `:inverse_of` option specifies the name of the `has_many` or `has_one` association that is the inverse of this association. Does not work in combination with the `:polymorphic` options.

```
class Customer < ActiveRecord::Base
  has_many :orders, inverse_of: :customer
end

class Order < ActiveRecord::Base
  belongs_to :customer, inverse_of: :orders
end
```

4.1.2.7 `:polymorphic`

Passing `true` to the `:polymorphic` option indicates that this is a polymorphic association. Polymorphic associations were discussed in detail earlier in this guide.

4.1.2.8 `:touch`

If you set the `:touch` option to `:true`, then the `updated_at` or `updated_on` timestamp on the associated object will be set to the current time whenever this object is saved or destroyed:

```
class Order < ActiveRecord::Base
  belongs_to :customer, touch: true
end

class Customer < ActiveRecord::Base
  has_many :orders
end
```

In this case, saving or destroying an order will update the timestamp on the associated customer. You can also specify a particular timestamp attribute to update:

```
class Order < ActiveRecord::Base
  belongs_to :customer, touch: :orders_updated_at
end
```

4.1.2.9 `:validate`

If you set the `:validate` option to `true`, then associated objects will be validated whenever you save this object. By default, this is `false`: associated objects will not be validated when this object is saved.

4.1.2.10 `:optional`